LAPORAN UTS APPL

Nama : Jessica Pasaribu

NIM : 41425079

Prodi : D4 Teknologi Rekayasa Perangkat Lunak

FAKULTAS VOKASI

INSTITUT TEKNOLOGI DEL

CASE STUDI 1

BAB 1 DESIGN THINKING & ARCHITECTURE SELECTION

1.1 Latar Belakang

MediTrack merupakan platform digital healthcare yang menghubungkan pasien, dokter, dan apotek dalam satu ekosistem terintegrasi. Sistem ini harus mampu menangani berbagai fitur penting seperti manajemen pengguna, penjadwalan janji temu, rekam medis elektronik (EHR), integrasi farmasi, sistem pembayaran, serta analitik.

Dengan ekspektasi pertumbuhan yang cepat dan adopsi lintas wilayah, sistem harus dirancang dengan mempertimbangkan:

1. Skalabilitas tinggi
2. Keamanan data medis
3. Maintainability
4. Kemudahan pengembangan lanjutan

1.2 Pemilihan Arsitektur Sistem

Berdasarkan kebutuhan sistem dan ekspektasi pertumbuhan, arsitektur yang dipilih adalah Hybrid Architecture, yaitu kombinasi antara Microservices Architecture dan Domain-Driven Design (DDD).

Pendekatan ini memanfaatkan keunggulan microservices dalam hal skalabilitas dan fleksibilitas, serta prinsip DDD untuk memastikan bahwa setiap bagian sistem dibangun berdasarkan domain bisnis yang jelas dan terstruktur.

Dalam arsitektur ini, sistem dibagi menjadi beberapa layanan (services) independen seperti User Service, Appointment Service, EHR Service, Pharmacy Service, Payment Service, dan Analytics Service. Setiap service merepresentasikan satu domain utama dan beroperasi secara mandiri dengan tanggung jawab yang spesifik.

1.3 Justifikasi Arsitektur

Pemilihan arsitektur ini didasarkan pada beberapa prinsip utama sebagai berikut:

1. **Separation of Concerns (SoC)**

   Setiap layanan dipisahkan berdasarkan domain bisnis:

   - User Service
   - Appointment Service
   - EHR Service
   - Pharmacy Service
   - Payment Service

2. **Scalability (Skalabilitas)**

   Setiap service dapat di skalakan secara independent

   - Appointment service bisa di scale saat traffic tinggi
   - Analytics bisa di opsimasi tanpa mengganggu service lain

3. **Maintainability & Modularity**

   Perubahan pada satu service tidak mempengaruhi service lain:

   - Update HER tidak mengganggu payment
   - Debugging lebih mudah karena isolasi layanan

1.4 trade-offs & Limitations

Aspek	**Penjelasan**
Kompleksitas	Lebih kompleks dibanding monolithic
Network Overhead	Komunikasi antar service via HTTP
Deployment	Membutuhkan manajemen service lebih banyak
Data Consistency	Lebih sulit menjaga konsistensi antar service

1.5 Dukungan terhadap Sistem

Scalability

- Setiap service dapat di-scale secara independen
- Load balancing dapat diterapkan

Maintainability

- Codebase terpisah
- Mudah testing dan debugging

Extensibility

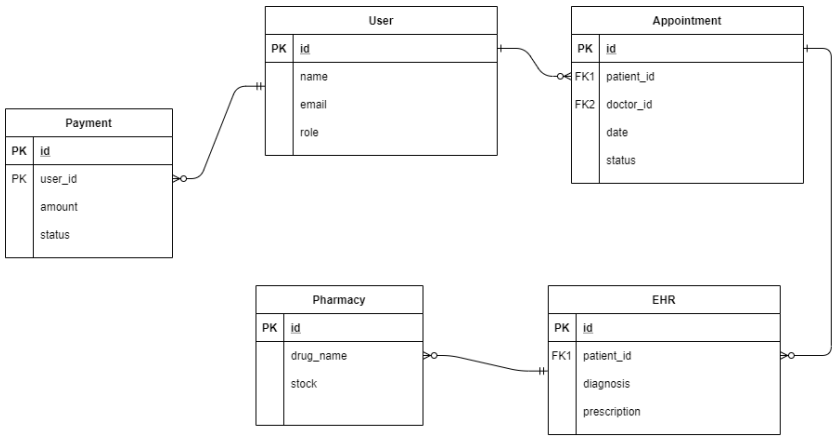
- Mudah menambahkan service baru (misal: AI diagnosis)
- Integrasi API eksternal lebih fleksibel

BAB II SYSTEM DECOMPOSITION & MODELING

2.1 Daftar Service

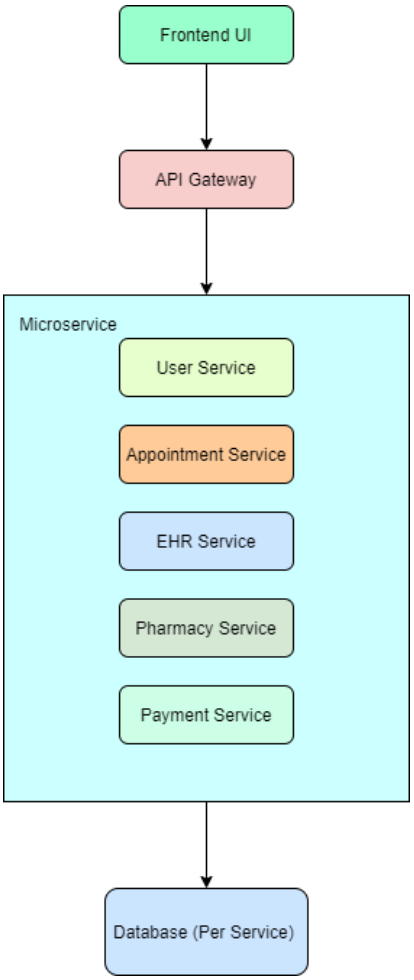
Service	Tanggung Jawab
User Service	Manajemen user (patient, doctor, admin)
Appointment Service	Booking, reschedule, cancel
EHR Service	Data medis pasien
Pharmacy Service	Obat & stok
Payment Service	Transaksi pembayaran
API Gateway	Routing & integrasi

2.2 Relationship



BAB 3 ARCHITECTURE VISUALIZATION

3.1 Komponen Sistem



3.2 Data Flow

1. User melakukan request dari UI
2. Request masuk ke API Gateway
3. Gateway meneruskan ke service terkait
4. Service memproses dan akses database
5. Response kembali ke UI

3.3 Boundary

Service Boundary:

- Setiap service memiliki database sendiri

API Boundary:

- Semua akses melalui API Gateway

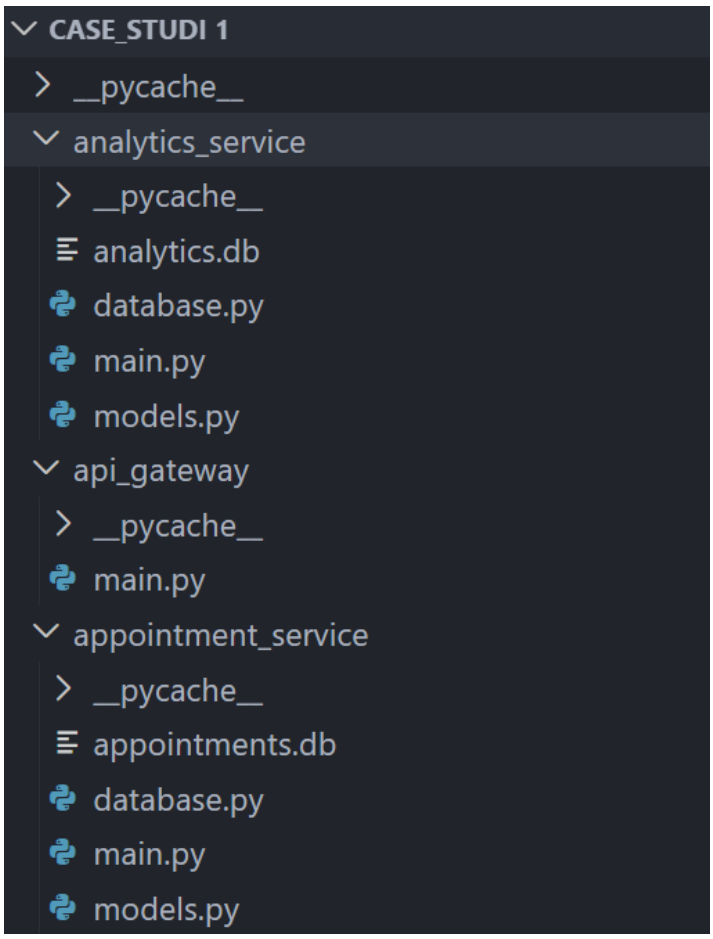
BAB 4 IMPLEMENTASI

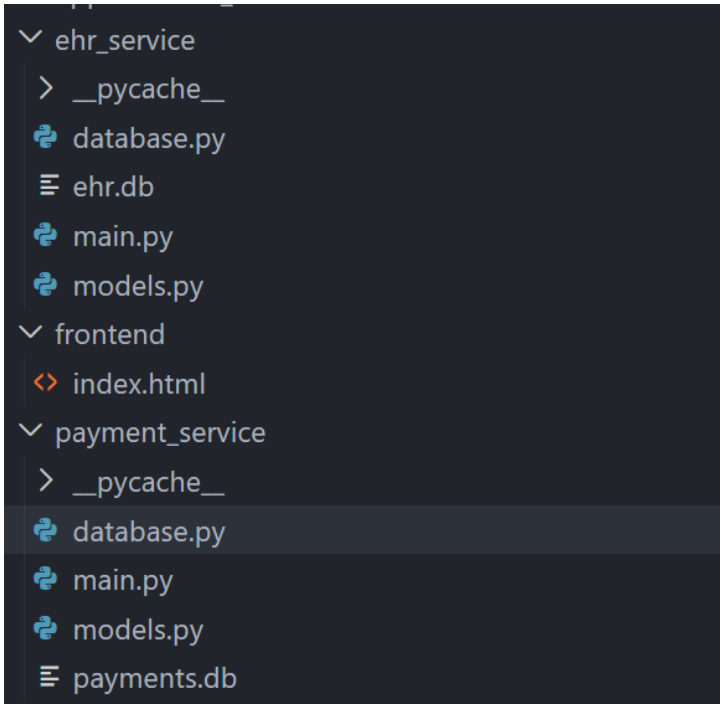
4.1 Teknologi

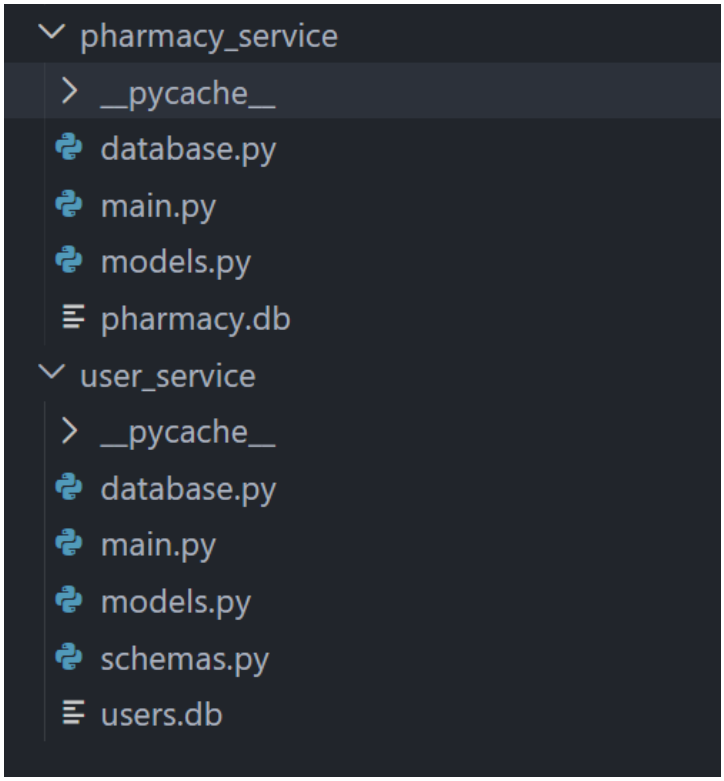
- Backend: FastAPI
- Database: SQLite
- ORM: SQLAlchemy
- Frontend: HTML + JavaScript
- API Gateway: FastAPI Proxy

4.2 Implementasi

Struktur Folder



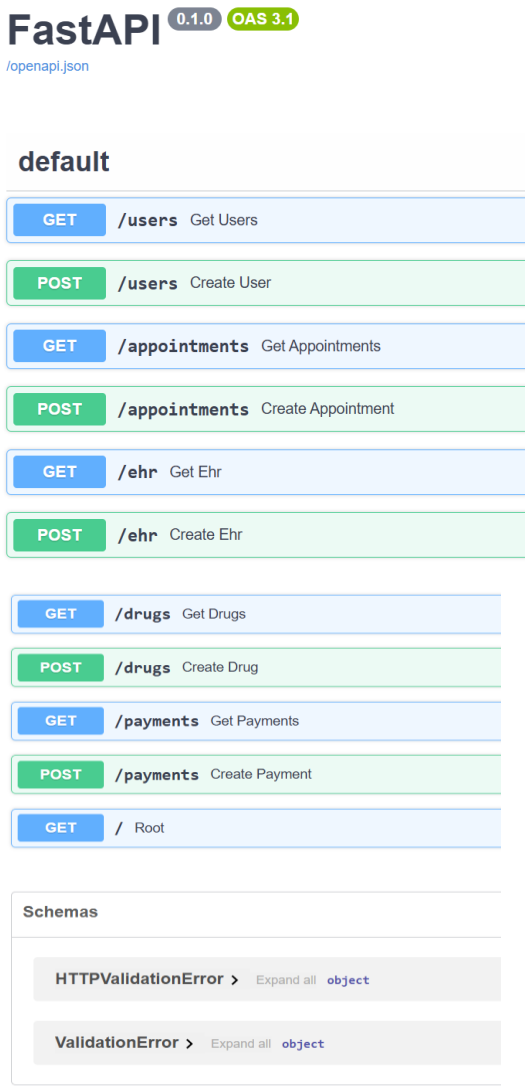

ehr_service
__pycache__
database.py
ehr.db
main.py
models.py
frontend
index.html
payment_service
__pycache__
database.py
main.py
models.py
payments.db


pharmacy_service
__pycache__
database.py
main.py
models.py
pharmacy.db
user_service
__pycache__
database.py
main.py
models.py
schemas.py
users.db

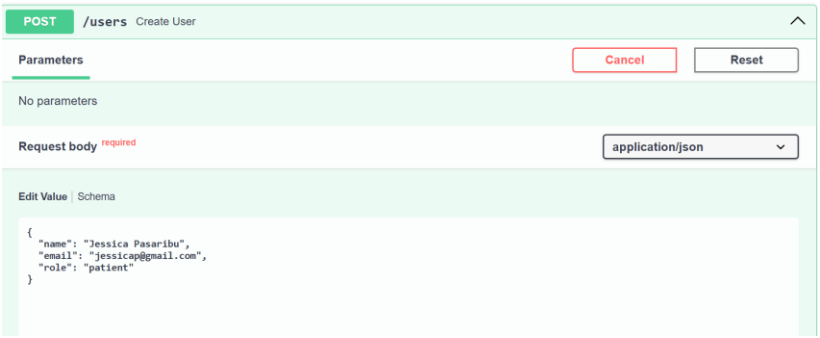
URL MASING-MASING SERVICE:

```
# SERVICE URL
USER_SERVICE = "http://localhost:8001"
APPOINTMENT_SERVICE = "http://localhost:8002"
EHR_SERVICE = "http://localhost:8003"
PHARMACY_SERVICE = "http://localhost:8004"
PAYMENT_SERVICE = "http://localhost:8005"
```

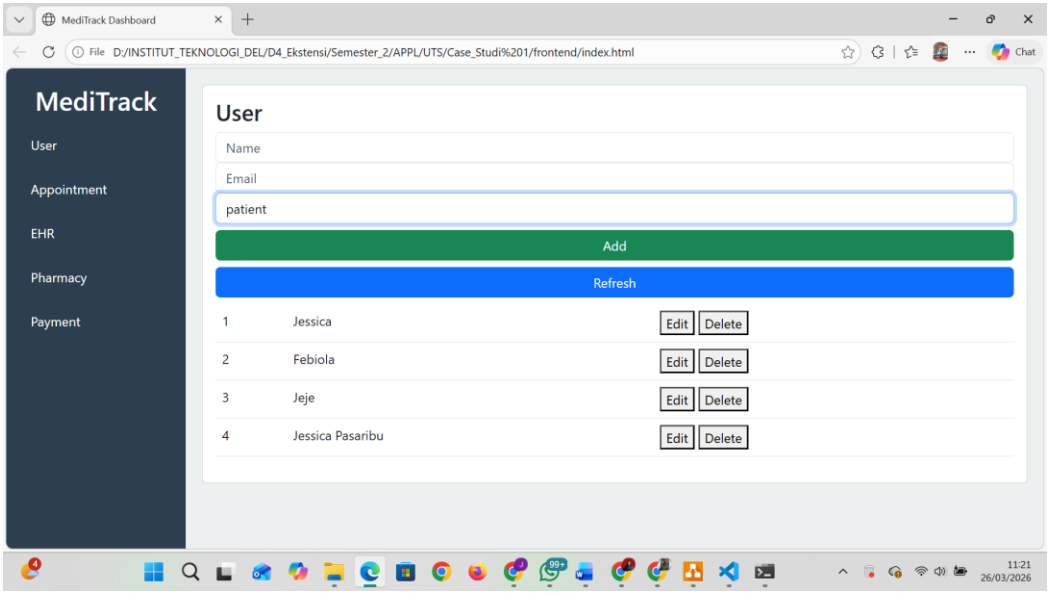
Implementasi Menggunakan FastAPI:



Contoh Create user:



Tampilan UI:

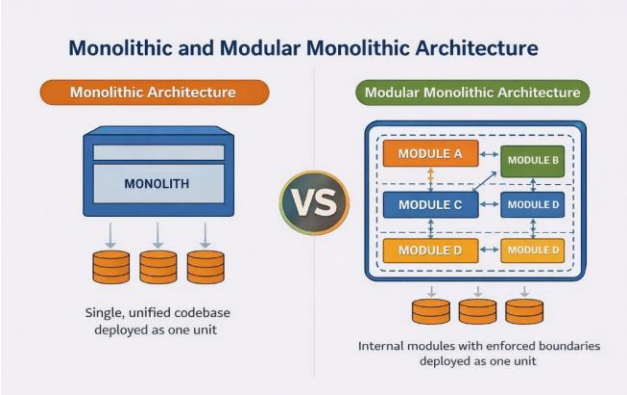


CASE STUDY 2

Use Case Utama

- **User registration & login**
    1. Pasien melakukan registrasi login
    2. Ditangani oleh Auth Module
- **Book Appointment**
    1. Pasien Membuat janji dengan dokter
    2. Sistem akan:
        - Validasi user
        - Proses pembayaran
- **Electronic Health Record (HER)**
    1. Menyimpan Riwayat medis pasien
    2. Ditangani oleh HER Module
- **Pharmacy**
    1. Pengelolaan obat
    2. Ditangani oleh Pharmacy Module
- **Payment**
    1. Mengelola Pembayaran Layanan
    2. Ditangani oleh Payment Module

Architecture Diagram



a. **Monolith Architecture**

Karakteristik:

1. Semua modul dalam satu aplikasi
2. Menggunakan 1 database
3. Komunikasi langsung antar module

b. **Microservices Architecture**

Karakteristik:

1. Setiap modul menjadi service terpisah
   - Auth Service
   - Appointment Service
   - Payment Service
   - HER Service
   - Pharmacy Service
2. Setiap service memiliki database sendiri
3. Komunikasi menggunakan REST API (HTTP)

c. **Coverage (Old -> New)**

Monolith Module	**Microservice**
Auth	Auth Service
Appointment	Appointment Service
EHR	HER Service
Pharmacy	Pharmacy Service
Payment	Payment Service

Semua fungsi lama tetap ada, hanya dipisah menjadi service

Reasoning & Trade-offs

Reasoning

Transformasi dilakukan karena:

- Monolith sulit dikembangkan
- Sulit scaling (harus scale semua)
- Tight coupling antar module

Microservices memberikan:

- Modularitas tinggi
- Independent deployment
- Scalability lebih baik

Trade-offs

Kelebihan Microservices:

- Scalability tinggi
- Deployment independen
- Fault isolation (1 service down tidak semua mati)

Kekurangan:

- Kompleksitas tinggi
- Perlu network communication
- Debugging lebih sulit
- Butuh orchestration

Migration Strategy

Metode : Strangler Pattern

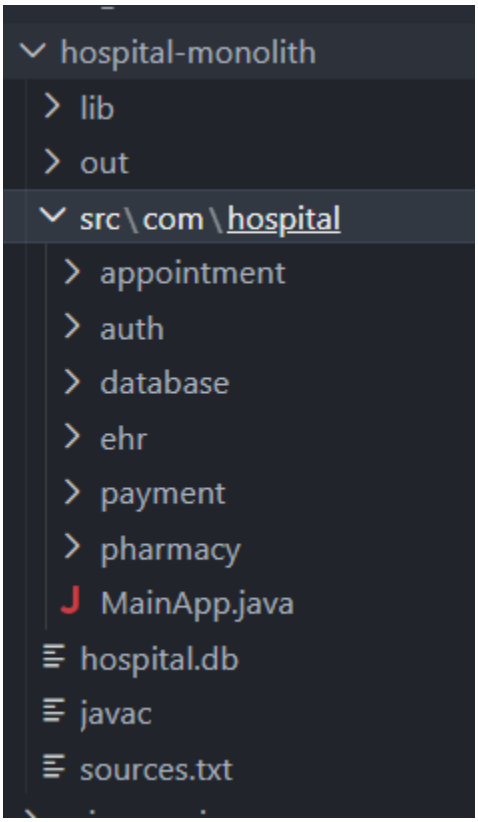
Tahapan:

1. Mulai dari monolith
2. Pisahkan service satu per satu:

    - Auth : Microservice
    - Payment : Microservice
3. Routing dialihkan ke service baru
4. Monolith perlahan ditinggalkan

Monolith Representation

Struktur



Karakteristik:

1. Semua modul dalam satu project
2. Menggunakan shared database
3. Komunikasi Langsung
4. Semua tergantung satu database

Output:

```
PROBLEMS    OUTPUT    TERMINAL    PORTS

TERMINAL
D:\INSTITUT_TEKNOLOGI_DEL\D4_Ekstensi\Semester_2\APPL\UTS\Case_Studi_2\hospital-monolith>java -cp ".;lib/sqlite-jdbc.jar;out" com.hospital.MainApp
Database initialized!

=== SYSTEM START ===
User registered
Login success!
Payment $100 processed for jessica
Appointment booked!
Medical record added
=== SYSTEM END ===

D:\INSTITUT_TEKNOLOGI_DEL\D4_Ekstensi\Semester_2\APPL\UTS\Case_Studi_2\hospital-monolith>
```

Microservices Mapping

a. Transformasi

Setiap module diubah menjadi service

Module	Service
Auth	Auth Service
Appointment	Appointment Service
EHR	EHR Service
Pharmacy	Pharmacy Service
Payment	Payment Service